Título: Apps, no chats. Modelos de IA locales corriendo en macOS.

Autor: Gorka Ludlow, Karma Digital, Montevideo, Uruguay, gorka@karma-digital.net

Resumen

La mayoría conoce la IA como un chat. Pero los modelos pueden correr como software real en tu propia computadora, haciendo un trabajo dentro de una app en lugar de responder mensajes en un chat. Esta charla recorre productos macOS ya publicados que hacen exactamente eso, con los modelos ejecutándose por completo en el dispositivo: transcripción y diarización de hablantes, comprensión de la pantalla mediante un servidor MCP, clonación de voz que lee texto con tu propia voz, y un asistente de realidad aumentada. Correr en local reduce el costo por inferencia, funciona sin conexión, y puede mantener tus datos privados, a cambio de pelear con el tamaño del modelo, la memoria, la latencia y el empaquetado en macOS. Compartimos la ingeniería honesta detrás de productos que ya están en manos de usuarios: qué modelos elegimos y por qué, qué funcionó, qué se rompió, y cuándo lo local es la decisión equivocada. Apps, no chats.

Abstract

Most people meet AI as a chat box. But the models can run as real software on your own machine, doing a job inside an app instead of answering prompts in a thread. This talk walks through shipping macOS products that do exactly that, with the models running entirely on-device: transcription and speaker diarization, screen understanding through an MCP server, voice cloning that reads text in your own voice, and an augmented-reality assistant. Running locally reduces per-inference cost, works offline, and can keep your data private, in exchange for fighting model size, memory, latency, and macOS packaging. We share the honest engineering behind products already in users' hands: which models we chose and why, what worked, what broke, and when local is the wrong call. Apps, not chats.

I. Introducción

Para la mayoría, “usar IA” es escribir en un chat y leer las respuestas. Es conveniente y práctico pero no es todo lo que se puede hacer. Hay una idea más simple: los mismos modelos pueden correr como software común, en tu propia computadora, haciendo bien un solo trabajo dentro de una app en lugar de sostener una conversación.

Correr un modelo en local es atractivo por razones concretas. Puede mantener tus datos en el dispositivo, reduce costos porque en local no hay que pagar por usar una API, y sigue funcionando sin conexión. Pero también es más difícil que hacer llamadas a endpoints. Los modelos ocupan mucho espacio en disco y usan mucha memoria (GPU). Éstas son consideraciones a tener para ver si vale la pena local.

Este trabajo describe productos macOS que ponen a trabajar modelos locales como apps enfocadas: transcripción y diarización de hablantes, comprensión de la pantalla mediante un servidor MCP, clonación de voz que lee texto con tu propia voz, y un asistente de realidad aumentada. En lugar de un banco de pruebas, describimos qué construimos y publicamos, y las decisiones y errores de ingeniería detrás de cada uno.

El artículo se organiza así. La Sección II describe el panorama y los retos de la inferencia en el dispositivo. La Sección III explica el enfoque detrás de cada producto. La Sección IV presenta resultados prácticos. La Sección V concluye, incluido cuándo lo local es la decisión equivocada.

II. Antecedentes, Retos y Problemas

La inferencia en el dispositivo se volvió práctica porque hoy caben modelos capaces en hardware de consumo. El reconocimiento de voz corre localmente con Whisper y con la familia Parakeet de NVIDIA; modelos pequeños de visión y de lenguaje corren en la GPU o el Neural Engine de una laptop; y la síntesis de voz, incluida la clonación de voz, ya no necesita un servidor. Las herramientas maduraron en paralelo: Core ML y Metal de Apple para la aceleración, y el Model Context Protocol (MCP) para conectar un modelo con el contexto de una aplicación.

El atractivo es la privacidad, el costo y el uso sin conexión, pero las restricciones son reales. Los modelos son grandes, la memoria en la computadora del usuario es finita, la latencia depende de hardware que uno no controla, y una app de escritorio debe empaquetarse, firmarse, notarizarse y mantenerse actualizada. La mayoría de los productos de IA esquivan todo esto llamando a una API alojada, que cambia esas restricciones por un costo por petición, una dependencia de red, y el envío de los datos del usuario fuera del dispositivo.

Los productos que se describen aquí toman el camino opuesto. Cada uno integra un modelo en una app de escritorio enfocada en lugar de una interfaz de chat, lo ejecuta en la computadora del usuario, y trata esas restricciones de ingeniería como el problema central a resolver. Este es un relato práctico de haberlo hecho en varios productos ya publicados, no un estudio comparativo de modelos.

III. Metodología Aplicada

El enfoque es el mismo en todos los productos: tomar una tarea concreta, mantener el modelo en la computadora del usuario, e integrarlo en una app enfocada. No se describe aquí la implementación interna de cada producto; se describen las decisiones que se repiten.

Dos formas de correr en local. En algunos productos la app incluye el modelo y lo ejecuta ella misma (por ejemplo, voz a texto o clonación de voz). En otros la app no trae modelo: es agnóstica y se conecta por MCP al modelo que el usuario elija, incluido uno local, al que le da un sentido nuevo como ver la pantalla. En ambos casos la inferencia ocurre en el dispositivo del usuario.

Elegir el modelo por la tarea, no al revés. Cuando la app incluye el modelo, se toma el más chico que resuelva bien la tarea: cabe en memoria, responde más rápido y es más fácil de empaquetar. Cuando la app es agnóstica, esa elección queda del lado del usuario y la app se diseña para no depender de un modelo en particular.

Diseñar alrededor de las restricciones del usuario. El hardware es el del usuario, no el nuestro: memoria finita, latencia variable, y una app que debe empaquetarse, firmarse, notarizarse y actualizarse en macOS. Esas restricciones se tratan como parte del diseño desde el principio, no como un problema del final.

IV. Resultados

Los resultados se presentan como lo que reciben los usuarios, no como detalles internos de implementación.

Voz. A partir de una grabación corta hecha dentro de la app, el sistema genera habla con la voz del usuario, con calidad de publicación y sin depender del micrófono con el que se grabó. Todo el proceso corre en la computadora del usuario.

Transcripción. La transcripción es precisa y separa a los hablantes, y funciona por completo sin conexión; el audio nunca sale del dispositivo.

Visión de pantalla. Cualquier modelo compatible con MCP, incluido uno local, puede ver la pantalla del usuario a demanda y responder con ella a la vista, sin subir la imagen a un servicio externo.

En los tres casos el resultado es el mismo patrón: una tarea concreta resuelta con el modelo corriendo en local, y los datos quedándose en el dispositivo.

Tabla I. Qué entrega cada producto:

Producto/tarea	Corre en local	Sin conexión	Datos en el dispositivo
Transcripción	sí	sí	sí
Clonación de voz	sí	sí	sí
Visión de pantalla	sí	según el modelo	según el modelo

V. Conclusiones

Los modelos locales no son solo una demostración: pueden ser la base de un producto que se publica y se usa. En lugar de una interfaz de chat, cada modelo hace un trabajo concreto dentro de una app, corriendo en la computadora del usuario. Eso reduce el costo por inferencia, funciona sin conexión y puede mantener los datos del usuario privados, siempre que se diseñe alrededor de las restricciones de su hardware.

Surgieron dos patrones. En uno, la app incluye y ejecuta el modelo. En otro, la app es agnóstica y le da un sentido nuevo, como ver la pantalla, al modelo que el usuario conecte, incluido uno local. Ambos mantienen la inferencia en el dispositivo.

Lo local no siempre es la respuesta correcta. Hay un techo de tamaño y de calidad: algunas tareas todavía superan lo que el hardware de un usuario puede correr bien, y en el patrón agnóstico la privacidad y el uso sin conexión dependen del modelo que elija el usuario. Además, empaquetar, firmar, notarizar y actualizar una app de escritorio es un costo real que hay que asumir.

VI. Biografía

Gorka Ludlow es cofundador de Karma Digital, donde construye y publica apps de IA para macOS que ejecutan modelos en el dispositivo, y ofrece consultoría en IA. LinkedIn: linkedin.com/in/gorka-ludlow

Referencias

[1] A. Radford, J. W. Kim, T. Xu, et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” arXiv:2212.04356, 2022. https://arxiv.org/abs/2212.04356 [2] NVIDIA, “Parakeet TDT 0.6B V2 (En),” model card, Hugging Face. https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2 [3] Anthropic, “Model Context Protocol,” 2024. https://modelcontextprotocol.io [4] Apple, “Core ML,” Apple Developer Documentation. https://developer.apple.com/documentation/coreml